

# Fase 9 — Diseño de la Arquitectura Distribuida

MGTI. Baltazar Martinez Galla

Continuación directa de la Fase 8. Ambos nodos tienen MariaDB 10.11 instalado con el esquema `lab_bdd` poblado con datos de prueba y snapshot `fase08-completa` tomado. Esta fase **no instala ni configura nada nuevo** en las máquinas virtuales: es una fase de análisis y diseño en la que se toma el esquema centralizado existente y se elabora el plan completo de distribución que se implementará en las Fases 10 a 16. El principal entregable es el **Documento de Diseño Distribuido (DDD)**, que servirá como referencia técnica en todas las fases posteriores.

## A. Objetivos de aprendizaje

Al finalizar esta fase, el estudiante será capaz de:

1. Aplicar la metodología **top-down** de diseño de bases de datos distribuidas para traducir un esquema relacional centralizado en un plan de distribución formal.
2. Distinguir y construir los tres esquemas del diseño distribuido: **esquema global**, **esquema de fragmentación** y **esquema de asignación**.
3. Definir fragmentos horizontales mediante **predicados simples** y combinaciones de **predicados minterm**, verificando su correctitud (completitud, disjunción y reconstrucción).
4. Definir fragmentos verticales usando el criterio de **afinidad de columnas** y verificar su correctitud (completitud y reconstrucción mediante JOIN sobre la PK).
5. Diseñar la estrategia de **fragmentación híbrida** que emerge de la combinación de las fragmentaciones horizontal y vertical sobre distintas tablas del mismo esquema.
6. Justificar las **decisiones de asignación de datos** a nodos específicos usando criterios de localidad de acceso, balanceo de carga y co-localización de tablas relacionadas por claves foráneas.
7. Elaborar la **matriz de asignación completa** del laboratorio, especificando para cada fragmento el nodo destino y la fase de implementación.
8. Escribir y ejecutar consultas SQL que simulen, en el nodo centralizado actual, las operaciones de reconstrucción que el sistema distribuido realizará en fases posteriores, verificando que producen resultados correctos.
9. Producir el **Documento de Diseño Distribuido** ( `fase09-disenyo-distribuido.md` ) que actuará como especificación técnica de referencia para las Fases 10 a 16.

## B. Conceptos teóricos necesarios

### 1. Metodología top-down de diseño de BDD.

El diseño de una base de datos distribuida puede abordarse de dos formas:

- *Top-down*: se parte de los requisitos globales, se diseña el esquema centralizado y después se decide cómo distribuirlo. Es el enfoque de este laboratorio: el esquema `lab_bdd` ya existe como punto de partida centralizado y se redistribuirá.
- *Bottom-up*: se integran esquemas preexistentes en distintos nodos en un esquema global unificado. Se usa cuando los datos ya existen en múltiples sistemas heredados.

El proceso top-down involucra tres fases de diseño, que se desarrollan en esta fase del laboratorio:

|  Paso |  Nombre                                |  Pregunta que responde                      |
|----------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------|
| 1                                                                                      |  Diseño del esquema global            |  ¿Qué datos existen y cómo se relacionan?  |
| 2                                                                                      |  Diseño del esquema de fragmentación |  ¿Cómo se divide cada relación en partes? |
| 3                                                                                      |  Diseño del esquema de asignación    |  ¿Qué fragmento va a qué nodo?            |

### 2. Fragmentación horizontal.

Una fragmentación horizontal divide las **filas** de una relación `R` en subconjuntos llamados fragmentos `R1, R2, ..., Rn`, cada uno definido por un predicado de selección:

 Text

$$R_i = \sigma(\text{predicado}_i)(R)$$

- *Predicado simple*: condición sobre un único atributo; por ejemplo, `region = 'norte'`.

- *Predicado minterm*: combinación booleana completa de todos los predicados simples relevantes. Si hay  $n$  predicados simples  $p_1, p_2, \dots, p_n$ , existen hasta  $2^n$  minterms posibles; solo los minterms *no vacíos* generan fragmentos reales.

Para `region` con cuatro valores distintos, los predicados simples son `p1: region = 'norte'`, `p2: region = 'sur'`, `p3: region = 'este'`, `p4: region = 'oeste'`. Este laboratorio los agrupa en dos fragmentos de dos minterms cada uno, por limitación de nodos de sharding disponibles (dos: nodo04 y nodo05).

### 3. Condiciones de correctitud de la fragmentación.

Toda fragmentación debe cumplir tres condiciones para ser válida:

- **Complejitud** (*completeness*):  $\forall t \in R, \exists i \text{ tal que } t \in R_i$ . Ninguna fila puede perderse; toda tupla de la relación global debe aparecer en al menos un fragmento.
- **Disjunción** (*disjointness*):  $\forall i \neq j, R_i \cap R_j = \emptyset$ . En fragmentación horizontal, ninguna fila debe pertenecer a más de un fragmento. En fragmentación vertical, las columnas no clave no deben repetirse (la PK sí se duplica, y es obligatorio hacerlo para la reconstrucción).
- **Reconstrucción** (*reconstruction*): la relación global puede obtenerse a partir de sus fragmentos — mediante `UNION ALL` para fragmentación horizontal, o mediante `JOIN` sobre la PK para fragmentación vertical.

### 4. Fragmentación vertical.

Una fragmentación vertical divide las **columnas** de una relación `R`. Formalmente:

Text

$$V_i = \pi(C_i \cup \{PK\})(R)$$

Cada fragmento proyecta un subconjunto de columnas `Ci`, más la **clave primaria**, que siempre debe estar presente en todos los fragmentos para permitir la reconstrucción por JOIN. Los conjuntos de columnas no clave deben ser disjuntos:  $C_i \cap C_j = \emptyset$  para todo  $i \neq j$ .

La técnica de **afinidad de columnas** estima qué tan frecuentemente se acceden conjuntamente dos columnas. Las de alta afinidad entre sí se agrupan en el mismo fragmento. En la práctica del laboratorio se usa el criterio de *patrón de uso*: columnas que aparecen en consultas frecuentes (listados, búsquedas, cálculo de precios) se separan de columnas de descripción extensa/TEXT consultadas solo en pantallas de detalle.

## 5. Fragmentación híbrida.

La fragmentación híbrida combina ambas estrategias sobre el mismo esquema. En este laboratorio surge de forma natural: `clientes`, `pedidos` y `detalle_pedidos` se fragmentan **horizontalmente** por región (Fase 13), mientras que `productos` se fragmenta **verticalmente** por grupos de columnas (Fase 14). Las consultas que combinan ambas tablas en un mismo resultado (Fase 15) cruzan los dos tipos de fragmentación, constituyendo el escenario híbrido sin necesidad de introducir una tercera tabla de ejemplo.

## 6. Estrategias de asignación de datos.

Una vez definidos los fragmentos, se decide en qué nodo(s) reside cada uno:

- *Particionada*: cada fragmento existe en exactamente un nodo. Maximiza el uso del espacio disponible; la caída de un nodo hace inaccesibles sus fragmentos.
- *Replicada*: el fragmento (o la relación completa) se copia en varios nodos. Mejora disponibilidad y rendimiento de lectura; complica las escrituras (consistencia).
- *Híbrida*: algunos fragmentos son particionados y otros son replicados.

Este laboratorio usa el modelo híbrido: `lab_bdd` completa se *replica* entre nodo01 y nodo02 (Fase 10, replicación física), y sobre los nodos 04 y 05 se aplica *particionamiento horizontal* de algunas tablas (Fase 13).

## 7. Co-localización de tablas relacionadas.

Cuando dos tablas se unen frecuentemente mediante JOIN (como `pedidos` y `detalle_pedidos`), y ambas se van a fragmentar horizontalmente, la decisión de **co-localizar** sus fragmentos en el mismo nodo elimina los joins remotos entre nodos, que son la operación más costosa en un sistema distribuido (transferencia de datos por red). La regla práctica es: si existe una clave foránea entre dos tablas y la tabla hija puede fragmentarse derivadamente por el mismo predicado de la tabla padre, deben co-localizarse en el mismo nodo.

## 8. El nodo coordinador y el motor Spider.

En un sistema distribuido, algún nodo debe actuar como **coordinador de consultas globales**: recibe las consultas del cliente, las descompone en subconsultas dirigidas a los nodos con los fragmentos relevantes, recibe los resultados parciales y los combina en la respuesta final. En este laboratorio, `bdd-nodo06` desempeñará ese rol mediante el motor **Spider** de MariaDB — un motor de almacenamiento nativo de MariaDB que actúa como tabla puente hacia tablas remotas en otros servidores MariaDB. Desde el punto de vista del cliente, Spider hace que las tablas distribuidas en nodo04/nodo05 aparezcan como tablas locales en nodo06, implementando así la transparencia de fragmentación y ubicación.

## C. Procedimiento paso a paso

### Paso 1 — Iniciar `bdd-nodo01` y conectarse por SSH.

Solo se necesita un nodo para el análisis; ambos tienen el mismo esquema y datos.

### Paso 2 — Ejecutar el análisis estadístico de `lab_bdd`.

Recopilar métricas de distribución de datos y de tamaño de columnas que justificarán cada decisión de diseño con evidencia cuantitativa (sección D.2).

### Paso 3 — Diseñar y verificar la fragmentación horizontal.

Definir los predicados de los dos fragmentos de `clientes`, `pedidos` y `detalle_pedidos`; ejecutar las consultas de verificación de completitud, disjunción y reconstrucción sobre los datos reales (sección D.3).

### Paso 4 — Diseñar y verificar la fragmentación vertical de `productos`.

Definir los dos grupos de columnas, justificar la agrupación según el patrón de uso esperado y ejecutar la verificación de correctitud (sección D.4).

### Paso 5 — Simular las consultas distribuidas en el entorno centralizado.

Escribir las cuatro consultas que en las Fases 13–16 el coordinador ejecutará de forma distribuida y ejecutarlas localmente para confirmar que producen resultados correctos antes de dispersar los datos (sección D.5).

### Paso 6 — Revisar la topología de replicación.

Generar el diagrama de texto de la topología completa del laboratorio dentro del nodo, que se trasladará al documento de diseño (sección D.6).

### Paso 7 — Generar la matriz de asignación completa.

Visualizar como consulta SQL la tabla que relaciona cada fragmento con su nodo destino, su IP y su fase de implementación (sección D.7).

### Paso 8 — Crear el Documento de Diseño Distribuido en el host.

Generar el archivo `C:\LabBDD\Documentacion\fase09-disenyo-distribuido.md` que consolida el diseño completo y servirá como referencia obligatoria en las Fases 10–16 (sección D.8).

### Paso 9 — Apagar la VM y tomar el snapshot de cierre de fase.

Aunque no hubo cambios en el sistema operativo ni en el motor, el snapshot marca el estado verificado del laboratorio en el punto en que el diseño fue completado (D.9).

## D. Comandos completos

Los comandos marcados **(VM)** se ejecutan en una sesión SSH en `bdd-nodo01`. Los marcados **(host)** se ejecutan en PowerShell en Windows. Los bloques iniciados con `sudo mariadb` se ejecutan dentro del prompt del motor.

### D.1 Iniciar `bdd-nodo01` y conectarse (host)

 PowerShell

```
VBoxManage startvm "bdd-nodo01" --type headless
```

Esperar 20–30 segundos y conectarse:

 PowerShell

```
ssh bddadmin@192.168.56.101
```

## D.2 Análisis estadístico del esquema lab\_bdd (VM)

 Bash

```
sudo mariadb lab_bdd << 'EOF'

-- =====
-- BLOQUE 1: Estructura del esquema global
-- =====

-- Tablas, motor y tamaño aproximado
SELECT TABLE_NAME                AS tabla,
       ENGINE                      AS motor,
       TABLE_ROWS                 AS filas_aprox,
       ROUND((DATA_LENGTH + INDEX_LENGTH) / 1024.0, 1) AS total_KB
FROM   information_schema.TABLES
WHERE  TABLE_SCHEMA = 'lab_bdd'
ORDER BY DATA_LENGTH DESC;

-- Columnas por tabla (base para decisiones de fragmentación vertical)
SELECT TABLE_NAME                AS tabla,
       COLUMN_NAME                 AS columna,
       ORDINAL_POSITION            AS pos,
       DATA_TYPE                  AS tipo,
       CHARACTER_MAXIMUM_LENGTH    AS max_chars,
       IS_NULLABLE                 AS nutable,
       COLUMN_KEY                  AS clave
FROM   information_schema.COLUMNS
WHERE  TABLE_SCHEMA = 'lab_bdd'
ORDER BY TABLE_NAME, ORDINAL_POSITION;

-- Restricciones de clave foránea (determinan co-localización obligatoria)
SELECT CONSTRAINT_NAME            AS restriccion,
       TABLE_NAME                AS tabla_hijo,
       COLUMN_NAME                 AS columna_hijo,
       REFERENCED_TABLE_NAME       AS tabla_padre,
       REFERENCED_COLUMN_NAME      AS columna_padre
FROM   information_schema.KEY_COLUMN_USAGE
WHERE  TABLE_SCHEMA = 'lab_bdd'
       AND REFERENCED_TABLE_NAME IS NOT NULL
ORDER BY TABLE_NAME;

-- =====
-- BLOQUE 2: Distribución del atributo de fragmentación
-- =====

-- Distribución de clientes por región (atributo candidato
a fragmentación)
SELECT region,
       COUNT(*)                    AS num_clientes,
       ROUND(COUNT(*) * 100.0 / 20, 1) AS pct_del_total,
       GROUP_CONCAT(ciudad ORDER BY ciudad) AS ciudades
FROM   clientes
GROUP BY region
```

```

ORDER BY region;
-- Distribución de pedidos por región y estado
SELECT region,
       COUNT(*) AS num_pedidos,
       ROUND(SUM(total), 2) AS ingresos
FROM pedidos
GROUP BY region
ORDER BY region;
-- Líneas de detalle co-localizables con cada fragmento de pedidos
SELECT p.region,
       COUNT(DISTINCT p.id) AS pedidos,
       COUNT(dp.id) AS lineas_detalle,
       ROUND(SUM(dp.subtotal), 2) AS facturacion
FROM pedidos p
JOIN detalle_pedidos dp ON dp.pedido_id = p.id
GROUP BY p.region
ORDER BY p.region;
-- =====
-- BLOQUE 3: Análisis de columnas de 'productos'
--           (candidatas a fragmentación vertical)
-- =====
-- Longitud promedio por columna de tipo texto (identifica las "pesadas")
SELECT 'sku' AS columna, ROUND(AVG(LENGTH(sku)), 1) AS
bytes_prom FROM productos UNION ALL
SELECT 'nombre', ROUND(AVG(LENGTH(nombre)), 1)
FROM productos UNION ALL
SELECT 'categoria', ROUND(AVG(LENGTH(categoria)), 1)
FROM productos UNION ALL
SELECT 'descripcion', ROUND(AVG(LENGTH(descripcion)), 1)
FROM productos UNION ALL
SELECT 'ficha_tecnica', ROUND(AVG(LENGTH(ficha_tecnica)), 1)
FROM productos UNION ALL
SELECT 'imagen_url', ROUND(AVG(LENGTH(imagen_url)), 1)
FROM productos;
EOF

```

*Registrar los resultados de los tres bloques. Estos datos son la evidencia cuantitativa que respalda cada decisión de diseño en los pasos siguientes.*

## D.3 Fragmentación horizontal — definición y verificación de correctitud (VM)

 Bash

```
sudo mariadb lab_bdd << 'EOF'
-- =====
-- DEFINICIÓN DE FRAGMENTOS HORIZONTALES
-- =====
-- Atributo de fragmentación: region ENUM('norte','sur','este','oeste')
-- Número de fragmentos: 2 (limitado a los nodos de sharding disponibles)
--
-- frag_A → bdd-nodo04 (192.168.56.104): region IN ('norte', 'este')
-- frag_B → bdd-nodo05 (192.168.56.105): region IN ('sur', 'oeste')
--
-- Justificación del agrupamiento: 5 clientes por valor de región
-- (datos uniformes) → ambos fragmentos tendrán 10 filas de clientes,
-- produciendo un balanceo de carga perfecto.
-- =====
-- ----- Vista de cada fragmento -----
SELECT 'clientes_frag_A (norte+este)' AS fragmento,
       COUNT(*)                      AS filas,
       GROUP_CONCAT(DISTINCT region) AS regiones_incluidas
FROM clientes WHERE region IN ('norte', 'este');
SELECT 'clientes_frag_B (sur+oeste)' AS fragmento,
       COUNT(*)                      AS filas,
       GROUP_CONCAT(DISTINCT region) AS regiones_incluidas
FROM clientes WHERE region IN ('sur', 'oeste');
-- ----- VERIFICACIÓN DE CORRECTITUD - CLIENTES -----
-- 1. COMPLETITUD: suma de fragmentos debe igualar el total global
SELECT 'Compleitud clientes' AS condicion,
       (SELECT COUNT(*) FROM clientes) AS total_global,
       (SELECT COUNT(*) FROM clientes WHERE region IN ('norte','este')) +
       (SELECT COUNT(*) FROM clientes WHERE region IN ('sur','oeste'))
AS suma_fragmentos,
CASE
  WHEN (SELECT COUNT(*) FROM clientes) =
       (SELECT COUNT(*) FROM clientes WHERE region IN
('norte','este')) +
       (SELECT COUNT(*) FROM clientes WHERE region
IN ('sur','oeste'))
  THEN 'OK' ELSE 'FALLA'
END AS resultado;
-- 2. DISYUNCIÓN: ninguna fila puede pertenecer a ambos fragmentos
-- (se verifica buscando filas cuyo valor de region satisfaga
-- simultáneamente los dos predicados; debe devolver 0)
SELECT 'Disjunción clientes' AS condicion,
COUNT(*) AS
```

```

filas_en_interseccion,
    CASE WHEN COUNT(*) = 0 THEN 'OK' ELSE 'FALLA' END AS resultado
FROM clientes
WHERE region IN ('norte','este') AND region IN ('sur','oeste');
-- 3. RECONSTRUCCIÓN: UNION ALL de fragmentos produce la relación global
SELECT 'Reconstrucción clientes'          AS condicion,
       (SELECT COUNT(*) FROM clientes) AS filas_globales,
       COUNT(*)                        AS filas_reconstruidas,
       CASE
           WHEN (SELECT COUNT(*) FROM clientes) = COUNT(*)
           THEN 'OK' ELSE 'FALLA'
       END AS resultado
FROM (
    SELECT * FROM clientes WHERE region IN ('norte', 'este')
    UNION ALL
    SELECT * FROM clientes WHERE region IN ('sur', 'oeste')
) AS reconstruccion;
-- ----- VERIFICACIÓN CONDENSADA - PEDIDOS -----
SELECT 'Compleitud pedidos'              AS condicion,
       CASE
           WHEN (SELECT COUNT(*) FROM pedidos) =
                (SELECT COUNT(*) FROM pedidos WHERE region IN
 ('norte','este')) +
                (SELECT COUNT(*) FROM pedidos WHERE region
IN ('sur','oeste'))
           THEN 'OK' ELSE 'FALLA'
       END AS resultado
UNION ALL
SELECT 'Disjunción pedidos',
       CASE WHEN
           (SELECT COUNT(*) FROM pedidos
            WHERE region IN ('norte','este') AND region IN ('sur','oeste'))
= 0
       THEN 'OK' ELSE 'FALLA' END
UNION ALL
SELECT 'Reconstrucción pedidos',
       CASE WHEN (SELECT COUNT(*) FROM pedidos) =
                (SELECT COUNT(*) FROM pedidos WHERE region IN
 ('norte','este')) +
                (SELECT COUNT(*) FROM pedidos WHERE region
IN ('sur','oeste'))
       THEN 'OK' ELSE 'FALLA' END;
-- ----- DETALLE_PEDIDOS: fragmentación derivada -----
-- detalle_pedidos no tiene columna 'region' propia; se fragmenta
-- siguiendo al pedido al que pertenece (co-localización obligatoria
-- por la FK pedido_id → pedidos.id que impide joins remotos).
SELECT 'detalle_frag_A (pedidos norte+este)' AS fragmento,
       COUNT(*)                               AS filas

```

```

FROM detalle_pedidos dp
JOIN pedidos p ON p.id = dp.pedido_id
WHERE p.region IN ('norte', 'este');
SELECT 'detalle_frag_B (pedidos sur+oeste)' AS fragmento,
COUNT(*) AS filas
FROM detalle_pedidos dp
JOIN pedidos p ON p.id = dp.pedido_id
WHERE p.region IN ('sur', 'oeste');
-- Verificación de completitud de la fragmentación derivada
SELECT 'Completitud detalle_pedidos' AS
condicion,
(SELECT COUNT(*) FROM detalle_pedidos)
AS total_global,
(SELECT COUNT(*) FROM detalle_pedidos dp
JOIN pedidos p ON p.id = dp.pedido_id
WHERE p.region IN ('norte','este')) +
(SELECT COUNT(*) FROM detalle_pedidos dp
JOIN pedidos p ON p.id = dp.pedido_id
WHERE p.region IN ('sur','oeste')) AS
suma_fragmentos,
CASE
WHEN (SELECT COUNT(*) FROM detalle_pedidos) =
(SELECT COUNT(*) FROM detalle_pedidos dp
JOIN pedidos p ON p.id = dp.pedido_id
WHERE p.region IN ('norte','este')) +
(SELECT COUNT(*) FROM detalle_pedidos dp
JOIN pedidos p ON p.id = dp.pedido_id
WHERE p.region IN ('sur','oeste'))
THEN 'OK' ELSE 'FALLA'
END AS resultado;
EOF

```

## D.4 Fragmentación vertical de **productos** — definición y verificación (VM)

 Bash

```
sudo mariadb lab_bdd << 'EOF'
-- =====
-- DEFINICIÓN DE FRAGMENTOS VERTICALES
-- =====
-- Tabla fuente: productos (10 columnas de datos + PK 'id')
--
-- Criterio de agrupación: patrón de uso esperado
-- COLUMNAS OPERACIONALES (consulta frecuente):
--     id*, sku, nombre, categoria, precio, stock, fecha_creacion
--     → aparecen en listados, búsquedas y cálculos de precios
--     → Fragmento V_basico → bdd-nodo04 (192.168.56.104)
--
-- COLUMNAS DE DETALLE (consulta poco frecuente, volumen alto):
--     id*, sku, descripcion, ficha_tecnica, imagen_url, peso_kg
--     → aparecen solo al consultar la ficha completa de un producto
--     → Columnas TEXT: promedio de 60-170 bytes/fila (análisis D.2)
--     → Fragmento V_detalle → bdd-nodo05 (192.168.56.105)
--
-- Nota: 'id' (PK) se duplica en ambos fragmentos para reconstrucción
--       por JOIN. 'sku' (clave alternativa) también se duplica por
--       ser referencia natural de negocio usada en ambos contextos;
--       esto es una decisión de diseño deliberada, no una violación
--       de la disjunción (que aplica a columnas NO CLAVE).
-- =====
-- Vista previa del fragmento básico (7 columnas)
SELECT id, sku, nombre, categoria, precio, stock, fecha_creacion
FROM   productos
ORDER BY id;
-- Vista previa del fragmento detalle (6 columnas)
SELECT id, sku,
       SUBSTRING(descripcion, 1, 45) AS descripcion_preview,
       SUBSTRING(ficha_tecnica, 1, 45) AS ficha_preview,
       imagen_url,
       peso_kg
FROM   productos
ORDER BY id;
-- ---- VERIFICACIÓN DE CORRECTITUD – VERTICAL ----
-- 1. COMPLETITUD: todos los productos deben estar en ambos fragmentos
SELECT 'Completitud V_basico' AS condicion,
       (SELECT COUNT(*) FROM productos) AS total_global,
       COUNT(*) AS
filas_en_fragmento,
       CASE WHEN COUNT(*) = (SELECT COUNT(*) FROM productos)
           THEN 'OK' ELSE 'FALLA' END AS resultado
```

```

FROM (SELECT id, sku, nombre, categoria, precio, stock, fecha_creacion
      FROM productos) AS frag_basico
UNION ALL
SELECT 'Complejidad V_detalle',
      (SELECT COUNT(*) FROM productos),
      COUNT(*),
      CASE WHEN COUNT(*) = (SELECT COUNT(*) FROM productos)
            THEN 'OK' ELSE 'FALLA' END
FROM (SELECT id, sku, descripcion, ficha_tecnica, imagen_url, peso_kg
      FROM productos) AS frag_detalle;
-- 2. DISYUNCIÓN DE COLUMNAS NO CLAVE
--   V_basico  (no clave): nombre, categoria, precio,
stock, fecha_creacion
--   V_detalle (no clave): descripcion, ficha_tecnica,
imagen_url, peso_kg
--   Intersección = vacío → condición cumplida
SELECT 'Disyunción de columnas no clave' AS condicion,
      'nombre, categoria, precio, stock, fecha_creacion'
AS cols_V_basico,
      'descripcion, ficha_tecnica, imagen_url, peso_kg'
AS cols_V_detalle,
      'Sin superposición (OK)' AS resultado;
-- 3. RECONSTRUCCIÓN: JOIN por PK reconstituye la relación completa
SELECT 'Reconstrucción productos (JOIN en id)' AS condicion,
      (SELECT COUNT(*) FROM productos) AS filas_globales,
      COUNT(*) AS
filas_reconstruidas,
      CASE WHEN COUNT(*) = (SELECT COUNT(*) FROM productos)
            THEN 'OK' ELSE 'FALLA' END AS resultado
FROM (
      SELECT b.id, b.sku, b.nombre, b.categoria, b.precio, b.stock,
            d.descripcion, d.ficha_tecnica, d.imagen_url, d.peso_kg,
            b.fecha_creacion
      FROM (SELECT id, sku, nombre, categoria, precio,
stock, fecha_creacion
            FROM productos) b
      JOIN (SELECT id, sku, descripcion, ficha_tecnica, imagen_url, peso_kg
            FROM productos) d ON b.id = d.id
) AS reconstruccion;
-- Verificación visual: primer registro reconstruido vs. original
SELECT 'Original' AS origen, id, sku, nombre, precio,
      SUBSTRING(descripcion, 1, 35) AS desc_preview
FROM productos WHERE id = 1
UNION ALL
SELECT 'Reconstruido' AS origen, b.id, b.sku, b.nombre, b.precio,
      SUBSTRING(d.descripcion, 1, 35)
FROM (SELECT id, sku, nombre, precio FROM productos WHERE id = 1) b
JOIN (SELECT id, descripcion FROM productos WHERE id = 1) d ON

```

```
b.id = d.id;  
EOF
```

## D.5 Simulación de consultas distribuidas en el entorno centralizado (VM)

*Estas consultas representan las operaciones que **bdd-nodo06 (coordinador)** ejecutará en las Fases 14–16 mediante el motor Spider. Ejecutarlas ahora — cuando todos los datos conviven en **bdd-nodo01** — permite verificar que los resultados esperados son correctos antes de dispersar los datos entre nodos.*

```

sudo mariadb lab_bdd << 'EOF'
-- =====
-- CONSULTA DISTRIBUIDA 1 – Pedidos con datos de cliente
-- Fragmentos: clientes_frag_A + pedidos_frag_A (nodo04)
--              clientes_frag_B + pedidos_frag_B (nodo05)
-- El coordinador envía la subconsulta a cada shard y hace
-- UNION ALL del resultado. No hay transferencia de datos entre
-- nodo04 y nodo05 porque los fragmentos de clientes y pedidos
-- están co-localizados por región.
-- =====
SELECT c.region,
       CONCAT(c.nombre, ' ', c.apellido) AS cliente,
       p.id                               AS pedido_id,
       p.estado,
       p.total
FROM   clientes c
JOIN   pedidos  p ON p.cliente_id = c.id
WHERE  c.region IN ('norte', 'este')      -- → subconsulta a nodo04
UNION ALL
SELECT c.region,
       CONCAT(c.nombre, ' ', c.apellido),
       p.id, p.estado, p.total
FROM   clientes c
JOIN   pedidos  p ON p.cliente_id = c.id
WHERE  c.region IN ('sur', 'oeste')       -- → subconsulta a nodo05
ORDER BY region, pedido_id;
-- =====
-- CONSULTA DISTRIBUIDA 2 – Líneas de pedido con nombre de producto
-- Fragmentos: pedidos_frag_* (nodo04/05) + V_basico (nodo04)
-- Solo se necesita el fragmento básico de productos (precio y nombre);
-- V_detalle (nodo05) no interviene → el coordinador evita un join
-- innecesario al nodo que contiene columnas TEXT.
-- =====
SELECT p.region,
       p.id                AS pedido_id,
       pr.nombre            AS producto,
       pr.categoria,
       dp.cantidad,
       dp.precio_unitario  AS precio_vendido,
       dp.subtotal
FROM   detalle_pedidos dp
JOIN   pedidos            p  ON  p.id = dp.pedido_id
JOIN   productos          pr ON pr.id = dp.producto_id
ORDER BY p.region, p.id;
-- =====
-- CONSULTA DISTRIBUIDA 3 – Ficha completa de un producto

```

```

-- Fragmentos: V_basico (nodo04) + V_detalle (nodo05)
-- El coordinador hace JOIN entre los dos fragmentos verticales
-- para reconstruir la fila completa. Solo se activa cuando el
-- usuario necesita ver la descripción y ficha técnica.
-- =====
SELECT b.id, b.sku, b.nombre, b.categoria,
       b.precio, b.stock,
       d.descripcion,
       d.ficha_tecnica,
       d.peso_kg
FROM   (SELECT id, sku, nombre, categoria, precio, stock
        FROM productos WHERE id = 1) b
JOIN   (SELECT id, descripcion, ficha_tecnica, peso_kg
        FROM productos WHERE id = 1) d ON b.id = d.id;
-- =====

-- CONSULTA DISTRIBUIDA 4 – Híbrida: ventas por región con
-- categorías de productos (cruza fragmentación horizontal de
-- clientes/pedidos y fragmentación vertical de productos)
-- =====
SELECT c.region,
       COUNT(DISTINCT c.id)           AS clientes_activos,
       COUNT(DISTINCT p.id)           AS pedidos,
       GROUP_CONCAT(DISTINCT pr.categoria
                    ORDER BY pr.categoria) AS categorias_compradas,
       ROUND(SUM(dp.subtotal), 2)      AS facturacion_total
FROM   clientes      c
JOIN   pedidos        p  ON  p.cliente_id = c.id
JOIN   detalle_pedidos dp ON dp.pedido_id = p.id
JOIN   productos      pr ON pr.id = dp.producto_id
GROUP BY c.region
ORDER BY facturacion_total DESC;
-- =====

-- RESUMEN: métricas que el diseño debe conservar íntegras
-- =====
SELECT 'clientes total'           AS metrica, COUNT(*) AS valor FROM clientes
UNION ALL
SELECT 'clientes frag_A',         COUNT(*)           FROM clientes
WHERE region IN ('norte','este') UNION ALL
SELECT 'clientes frag_B',         COUNT(*)           FROM clientes
WHERE region IN ('sur','oeste')  UNION ALL
SELECT 'pedidos total',           COUNT(*)           FROM pedidos
UNION ALL
SELECT 'pedidos frag_A',          COUNT(*)           FROM pedidos
WHERE region IN ('norte','este') UNION ALL
SELECT 'pedidos frag_B',          COUNT(*)           FROM pedidos
WHERE region IN ('sur','oeste')  UNION ALL
SELECT 'detalle total',           COUNT(*)           FROM
detalle_pedidos UNION ALL

```

```
SELECT 'productos total (V_basico)', COUNT(*) FROM productos
UNION ALL
SELECT 'productos total (V_detalle)', COUNT(*)
FROM productos;
EOF
```

---

## D.6 Diagrama de topología de replicación (VM — referencia para el DDD)

 Bash

```
cat > /tmp/topologia_lab_bdd.txt << 'EOF'
=====
TOPOLOGÍA COMPLETA DEL LABORATORIO BDD — REFERENCIA FASE 9
=====
REPLICACIÓN FÍSICA — Fase 10


---


bdd-nodo01 [MAESTRO | 192.168.56.101]
  lab_bdd (esquema completo)
  Binary log activo, GTID habilitado
  Acepta lecturas Y escrituras
    |
    | binlog stream (row-based, GTID)
    ▼
bdd-nodo02 [ESCLAVO | 192.168.56.102]
  lab_bdd (réplica completa, solo lectura)
  read_only = ON
REPLICACIÓN LÓGICA / MULTI-MAESTRO — Fase 11


---


bdd-nodo03 [MULTI-MAESTRO | 192.168.56.103]
  lab_bdd (réplica completa)
  Acepta lecturas Y escrituras
  Replicación bidireccional con nodo01
FRAGMENTACIÓN HORIZONTAL — Fase 13


---


bdd-nodo04 [SHARD-A | 192.168.56.104]
  clientes      WHERE region IN ('norte','este') → 10 filas
  pedidos      WHERE region IN ('norte','este') → 10 filas
  detalle_pedidos co-localizado con pedidos_frag_A → ~18 filas
bdd-nodo05 [SHARD-B | 192.168.56.105]
  clientes      WHERE region IN ('sur','oeste') → 10 filas
  pedidos      WHERE region IN ('sur','oeste') → 10 filas
  detalle_pedidos co-localizado con pedidos_frag_B → ~18 filas
FRAGMENTACIÓN VERTICAL — Fase 14


---


bdd-nodo04: V_productos_basico
  Columnas: id*, sku, nombre, categoria, precio, stock, fecha_creacion
bdd-nodo05: V_productos_detalle
  Columnas: id*, sku, descripcion, ficha_tecnica, imagen_url, peso_kg
COORDINADOR DE CONSULTAS DISTRIBUIDAS — Fases 14-16


---


bdd-nodo06 [COORDINADOR | 192.168.56.106]
  Motor Spider habilitado
  Tablas tipo SPIDER apuntando a nodo04 y nodo05
  Recibe consultas globales del cliente, las descompone
```

```
y combina resultados (no almacena datos de lab_bdd)
CLIENTE EXTERNO (opcional) – Fases 16-17
```

---

```
bdd-cliente [192.168.56.107]
  Conecta solo a nodo06 (ignora la topología interna)
  Prueba de transparencia de distribución
=====
EOF
cat /tmp/topologia_lab_bdd.txt
```

## D.7 Matriz de asignación completa (VM — visualización SQL)

 Bash

```
sudo mariadb << 'EOF'
-- Tabla virtual de referencia: matriz de asignación del laboratorio
SELECT fragmento, tabla_origen, tipo_frag,
       predicado_o_columnas, nodo_destino, ip_destino, fase
FROM (
  SELECT 'lab_bdd_completa (maestro)' AS fragmento,
        'todas' AS tabla_origen,
        'REPLICADA' AS tipo_frag,
        'esquema completo' AS predicado_o_columnas,
        'bdd-nodo01' AS nodo_destino,
        '192.168.56.101' AS ip_destino,
        '10' AS fase
  UNION ALL SELECT 'lab_bdd_completa (esclavo)', 'todas', 'REPLICADA',
        'esquema completo', 'bdd-nodo02', '192.168.56.102', '10'
  UNION ALL SELECT 'lab_bdd_multimaestro', 'todas', 'REPLICADA',
        'esquema completo', 'bdd-nodo03', '192.168.56.103', '11'
  UNION ALL SELECT 'clientes_frag_A', 'clientes', 'HORIZONTAL',
        "region IN ('norte', 'este')", 'bdd-nodo04', '192.168.56.104', '13'
  UNION ALL SELECT 'clientes_frag_B', 'clientes', 'HORIZONTAL',
        "region IN ('sur', 'oeste')", 'bdd-nodo05', '192.168.56.105', '13'
  UNION ALL SELECT 'pedidos_frag_A', 'pedidos', 'HORIZONTAL',
        "region IN ('norte', 'este')", 'bdd-nodo04', '192.168.56.104', '13'
  UNION ALL SELECT 'pedidos_frag_B', 'pedidos', 'HORIZONTAL',
        "region IN ('sur', 'oeste')", 'bdd-nodo05', '192.168.56.105', '13'
  UNION ALL SELECT 'detalle_frag_A', 'detalle_pedidos', 'HORIZONTAL
(dervivada)',
        'JOIN pedidos frag_A', 'bdd-nodo04', '192.168.56.104', '13'
  UNION ALL SELECT 'detalle_frag_B', 'detalle_pedidos', 'HORIZONTAL
(dervivada)',
        'JOIN pedidos frag_B', 'bdd-nodo05', '192.168.56.105', '13'
  UNION ALL SELECT 'V_productos_basico', 'productos', 'VERTICAL',
        'id,sku,nombre,categoria,precio,stock,fecha_crea', 'bdd-
nodo04', '192.168.56.104', '14'
  UNION ALL SELECT 'V_productos_detalle', 'productos', 'VERTICAL',
        'id,sku,descripcion,ficha_tecnica,imagen_url,peso', 'bdd-
nodo05', '192.168.56.105', '14'
) AS matriz
ORDER BY fase, tipo_frag, tabla_origen;
EOF
```

## D.8 Crear el Documento de Diseño Distribuido en el host (host — PowerShell)

PowerShell

```
$ddd = @"
# Documento de Diseño Distribuido — Laboratorio BDD
Generado en la Fase 9. Referencia técnica obligatoria para las Fases 10-16.
Fecha de generación: $(Get-Date -Format 'yyyy-MM-dd HH:mm')
---
## 1. Esquema global
Base de datos : lab_bdd
Nodo de origen: bdd-nodo01 (192.168.56.101)
| Tabla          | Filas | Motor  | Atributo de distribución          |
|-----|-----|-----|-----|
| clientes       | 20    | InnoDB | region ENUM (fragmentación horiz.) |
| productos     | 10    | InnoDB | columnas (fragmentación vertical) |
| pedidos       | 20    | InnoDB | region ENUM (fragmentación horiz.) |
| detalle_pedidos | 36    | InnoDB | derivada de pedidos (co-localizada) |
---
## 2. Plan de replicación
### Fase 10 — Replicación física (maestro-esclavo)
| Nodo  | IP          | Rol      | Modo de acceso          |
|-----|-----|-----|-----|
| nodo01 | 192.168.56.101 | MAESTRO | Lectura + escritura    |
| nodo02 | 192.168.56.102 | ESCLAVO | Solo lectura (RO)      |
Método: binary log basado en GTID, replicación row-based.
### Fase 11 — Replicación lógica
| Nodo  | IP          | Rol          |
|-----|-----|-----|
| nodo03 | 192.168.56.103 | MULTI-MAESTRO |
---
## 3. Plan de fragmentación horizontal (Fase 13)
Atributo de fragmentación: region
| Fragmento          | Predicado          | Nodo  | IP |
| Filas |
|-----|-----|-----|-----|
---|-----|
| clientes_frag_A    | region IN ('norte','este') | nodo04 |
192.168.56.104 | 10 |
| clientes_frag_B    | region IN ('sur','oeste') | nodo05 |
192.168.56.105 | 10 |
| pedidos_frag_A     | region IN ('norte','este') | nodo04 |
192.168.56.104 | 10 |
| pedidos_frag_B     | region IN ('sur','oeste') | nodo05 |
192.168.56.105 | 10 |
| detalle_frag_A     | JOIN pedidos frag_A | nodo04 |
192.168.56.104 | ~18 |

```

```

| detalle_frag_B      | JOIN pedidos frag_B      | nodo05 |
192.168.56.105 | ~18 |
Correctitud verificada: completitud OK, disjunción OK, reconstrucción OK.
---
## 4. Plan de fragmentación vertical (Fase 14)
Tabla fuente: productos
| Fragmento          | Columnas          |
Nodo | IP          |
|-----|-----|
| V_productos_basico | id*, sku, nombre, categoria, precio, stock, fecha |
nodo04 | 192.168.56.104 |
| V_productos_detalle | id*, sku, descripcion, ficha_tecnica, imagen, peso |
nodo05 | 192.168.56.105 |
Clave de reconstrucción: JOIN en columna 'id'.
Correctitud verificada: completitud OK, disjunción de columnas OK,
reconstrucción OK.
---
## 5. Coordinador de consultas distribuidas (Fases 14-16)
Nodo      : bdd-nodo06 (192.168.56.106)
Tecnología: Motor Spider de MariaDB
Rol       : recibir consultas globales, descomponerlas en subconsultas
            dirigidas a nodo04/nodo05 y combinar resultados.
---
## 6. Transparencia de distribución objetivo
| Tipo de transparencia | Mecanismo          | Implementada
en |
|-----|-----|-----|
| Replicación          | Maestro-esclavo transparente | Fases 10-
11 |
| Fragmentación        | Spider oculta la partición    | Fase
14+ |
| Ubicación            | Spider oculta el nodo físico  | Fase
14+ |
| Concurrencia         | InnoDB MVCC + pruebas de fallo | Fase
17+ |
---
## 7. Historial de snapshots del laboratorio
| Snapshot          | Estado en ese punto |
|-----|-----|
| fase05-completa   | Ubuntu Server instalado, SSH activo |
| fase06-completa   | IP estática configurada, SSH verificado |
| fase07-completa   | MariaDB 10.11 instalado, utf8mb4 activo |
| fase08-completa   | lab_bdd creada y poblada, usuarios, slow log |
| fase09-completa   | Diseño distribuido definido y validado |
"@
$ruta = "C:\LabBDD\Documentacion\fase09-disenyo-distribuido.md"

```

```
$ddd | Out-File -FilePath $ruta -Encoding UTF8
Write-Host "DDD creado en: $ruta"
# Verificar
Get-Item $ruta | Select-Object Name, Length, LastWriteTime
```

## D.9 Apagar la VM y tomar los snapshots de cierre de fase (host)

Desde la sesión SSH abierta en `bdd-nodo01`:

```
Bash

sudo poweroff
```

Confirmar desde el host que la VM se detuvo:

```
PowerShell

VBoxManage list runningvms
```

La salida debe estar vacía. Tomar los snapshots:

```
PowerShell

VBoxManage snapshot "bdd-nodo01" take "fase09-completa" `
  --description "Diseño distribuido validado: frag. horizontal
(clientes/pedidos/detalle), vertical (productos). DDD generado
en Documentacion/"
VBoxManage snapshot "bdd-nodo02" take "fase09-completa" `
  --description "Sin cambios respecto a fase08-completa. Snapshot de hito de
diseño para coherencia de numeración."
```

**Nota:** `bdd-nodo02` no recibió cambios en esta fase, pero se toma el snapshot para mantener la coherencia de numeración entre ambos nodos, lo que simplifica la identificación del estado de cada VM en cualquier punto del historial.

Confirmar los snapshots:

```
PowerShell

VBoxManage snapshot "bdd-nodo01" list
VBoxManage snapshot "bdd-nodo02" list
```

Cada VM debe mostrar cinco snapshots: `fase05-completa` hasta `fase09-completa`.

---

## E. Verificación de funcionamiento

Esta fase se considera completa cuando se cumplen todos los puntos siguientes:

1. El análisis estadístico (D.2) se ejecutó sin errores y el estudiante puede interpretar los resultados: 5 clientes por región, distribución de tamaño de columnas TEXT de `productos` (descripcion y ficha\_tecnica:  $\approx 100-170$  bytes promedio; columnas operacionales:  $\approx 10-15$  bytes).
2. Las **tres condiciones de correctitud de la fragmentación horizontal** de `clientes` (D.3) devuelven `OK`:
  - Completitud: suma de fragmentos = 20.
  - Disjunción: filas en intersección = 0.
  - Reconstrucción: UNION ALL produce 20 filas.
3. Las mismas tres condiciones se verificaron para `pedidos` (resultado `OK`) y la completitud para `detalle_pedidos` (resultado `OK`, con 36 filas en total).
4. Las **condiciones de correctitud de la fragmentación vertical** de `productos` (D.4) devuelven `OK`:
  - Completitud: los 10 productos aparecen en `v_basico` y en `v_detalle`.
  - Disjunción de columnas: confirmada (sin columnas no clave compartidas).
  - Reconstrucción: JOIN por `id` produce 10 filas con todos los atributos.
5. Las **cuatro consultas distribuidas simuladas** (D.5) devuelven resultados no vacíos y coherentes; en particular, la Consulta 4 devuelve exactamente 4 filas (una por región) con datos de facturación correctos.
6. El diagrama de topología (D.6) se generó en `/tmp/topologia_lab_bdd.txt` y el estudiante puede describir el rol de cada nodo sin leerlo.
7. La consulta de la matriz de asignación (D.7) muestra 11 filas, una por cada fragmento planificado (3 replicadas + 6 horizontales + 2 verticales).
8. El archivo `C:\LabBDD\Documentacion\fase09-disenyo-distribuido.md` existe en el host con un tamaño mayor a cero y contiene las siete secciones del diseño.
9. El estudiante puede explicar de memoria:
  - Por qué `detalle_pedidos` se co-localiza con `pedidos` (no se fragmenta independientemente).
  - Por qué la PK `id` debe estar en **ambos** fragmentos verticales.
  - Por qué `bdd-nodo06` es el coordinador y no `bdd-nodo01`.

- La diferencia entre la fragmentación híbrida “en una sola tabla” y la que emerge de combinar tablas con distintos tipos de fragmentación.

10. Los snapshots `fase09-completa` existen en `bdd-nodo01` y `bdd-nodo02`.

---

## F. Problemas comunes y soluciones

| Problema                                                                                                     | Causa probable                                                                    | Solución                                                                                                                                                                                                                                        |
|--------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| La verificación de completitud devuelve <code>FALLA</code> para <code>clientes</code>                        | Los predicados IN no cubren todos los valores del dominio de <code>region</code>  | Verificar con <code>SELECT DISTINCT region FROM clientes</code> que los cuatro valores reales son exactamente <code>norte, sur, este, oeste</code> ; asegurarse de que los dos predicados formen una partición completa del dominio             |
| La verificación de disjunción devuelve un conteo > 0                                                         | Los predicados de los dos fragmentos comparten algún valor de <code>region</code> | Revisar que <code>{'norte','este'} ∩ {'sur','oeste'} = ∅</code> ; si se modificaron los predicados de ejemplo, confirmar que no hay valor compartido                                                                                            |
| La reconstrucción vertical cuenta menos filas que la tabla original                                          | El JOIN entre fragmentos excluye filas (posible valor NULL en la PK)              | Ejecutar <code>SELECT COUNT(*) FROM productos WHERE id IS NULL;</code> — en InnoDB la PK nunca es NULL; si el resultado es > 0, hay corrupción de datos que debe investigarse antes de continuar                                                |
| La consulta distribuida simulada (D.5) devuelve 0 filas                                                      | La sesión no estaba en el contexto de <code>lab_bdd</code> al ejecutar el bloque  | Asegurar que el comando inicia con <code>sudo mariadb lab_bdd</code> (base de datos especificada) o agregar <code>USE lab_bdd;</code> al inicio del bloque HERE-DOC                                                                             |
| <code>Out-File</code> en PowerShell genera el archivo con codificación BOM o con saltos de línea incorrectos | Comportamiento de PowerShell 5.x con <code>-Encoding UTF8</code>                  | En PowerShell 7+ usar <code>-Encoding UTF8NoBOM</code> ; en PowerShell 5.x, el BOM no afecta la legibilidad del archivo pero puede causar problemas si se edita en ciertos programas                                                            |
| El estudiante no puede justificar la elección de <code>region</code> como atributo de fragmentación          | No se analizaron los datos antes de tomar la decisión                             | Revisar el Bloque 2 del análisis estadístico (D.2): la distribución uniforme de 5 clientes por región confirma que <code>region</code> produce fragmentos balanceados; un atributo desbalanceado (ej. <code>ciudad</code> ) produciría hotspots |
| Hay dudas sobre si duplicar <code>sku</code> en ambos fragmentos verticales viola la disjunción              | Confusión entre columnas clave y no clave                                         | La disjunción aplica solo a columnas <b>no clave</b> ; <code>sku</code> es una clave alternativa (UNIQUE) y puede — y conviene — aparecer en ambos fragmentos para que las aplicaciones puedan referenciar un producto sin                      |

|                                                                       |                                                                                                                                |                                                                                                                                                                                                                                                         |
|-----------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                                                       |                                                                                                                                | necesitar un JOIN cuando solo saben el SKU. Documentarlo como decisión de diseño intencional                                                                                                                                                            |
| El snapshot de <code>bdd-nodo02</code> falla porque la VM está activa | <code>bdd-nodo02</code> quedó encendida desde una sesión de trabajo anterior                                                   | Ejecutar <code>VBoxManage list runningvms</code> ; si aparece, apagarla con <code>VBoxManage controlvm "bdd-nodo02" acpipowerbutton</code> antes de tomar el snapshot                                                                                   |
| La Consulta 4 (D.5) devuelve menos de 4 filas                         | Algún cliente de alguna región no tiene pedidos, o las regiones están mal escritas en <code>/etc/hosts</code> (no aplica aquí) | Verificar con <code>SELECT DISTINCT region FROM pedidos</code> que los cuatro valores existen; la Consulta 4 usa LEFT JOIN implícito vía JOIN, por lo que regiones sin pedidos no aparecerán — si se desea incluirlas, cambiar a <code>LEFT JOIN</code> |

## G. Checklist de validación

- ☐ Ejecuté el análisis estadístico completo (D.2) en `bdd-nodo01` y tomé nota de los resultados: distribución por región, tamaño de columnas TEXT, FKs que imponen co-localización.
- ☐ Identifiqué `region` como atributo de fragmentación horizontal de `clientes` y `pedidos`, con justificación cuantitativa (distribución uniforme, 5 filas por valor).
- ☐ Definí los predicados de los dos fragmentos: `frag_A = region IN ('norte','este')`, `frag_B = region IN ('sur','oeste')`.
- ☐ Las tres condiciones de correctitud de la fragmentación horizontal de `clientes` devuelven `OK`: completitud, disjunción y reconstrucción.
- ☐ Las mismas condiciones se verificaron para `pedidos` (OK) y la completitud para `detalle_pedidos` (OK, 36 filas).
- ☐ Definí los dos fragmentos verticales de `productos`: `v_basico` (columnas operacionales de alta frecuencia) y `v_detalle` (columnas TEXT de baja frecuencia).
- ☐ Las condiciones de correctitud de la fragmentación vertical devuelven `OK`: completitud en ambos fragmentos, disjunción de columnas no clave, reconstrucción por JOIN en `id`.
- ☐ Ejecuté las cuatro consultas distribuidas simuladas (D.5) y confirmé resultados correctos y no vacíos.
- ☐ Generé el diagrama de topología (D.6) y puedo describir el rol de cada nodo.
- ☐ Ejecuté la consulta de la matriz de asignación (D.7) y puedo explicar cada fila.
- ☐ El archivo `C:\LabBDD\Documentacion\fase09-disenyo-distribuido.md` existe y contiene las siete secciones del diseño completo.

- ☐ Apagué `bdd-nodo01` de forma ordenada y tomé el snapshot `fase09-completa` en `bdd-nodo01` y `bdd-nodo02`.
- ☐ Puedo explicar la diferencia entre fragmentación horizontal, vertical e híbrida usando ejemplos concretos del esquema `lab_bdd`.
- ☐ Puedo justificar por qué `detalle_pedidos` se co-localiza con `pedidos` en lugar de asignarse a un nodo independiente.
- ☐ Puedo explicar por qué la PK `id` debe duplicarse en los dos fragmentos verticales de `productos`.
- ☐ Puedo describir el rol de `bdd-nodo06` como coordinador y la tecnología que usará (motor Spider de MariaDB).

---

## Preguntas teóricas para estudiantes

1. La fragmentación horizontal de `clientes` agrupa `norte+este` en un fragmento y `sur+oeste` en otro, en lugar de hacer un fragmento por cada valor de `region` (cuatro fragmentos). Explica las ventajas e inconvenientes de ambas estrategias. ¿Qué criterio debería dominar la decisión: el número de nodos disponibles, el volumen de datos por fragmento, la localidad de las consultas más frecuentes o la facilidad de mantenimiento futuro del esquema de distribución?
2. La tabla `detalle_pedidos` no tiene columna `region` propia, pero el diseño la co-localiza con los fragmentos de `pedidos`. Explica en detalle por qué esta co-localización es correcta desde el punto de vista del rendimiento de las consultas distribuidas. ¿Qué problema concreto y medible (en términos de tráfico de red y latencia) surgiría si `detalle_pedidos` se asignara a un único nodo centralizado, independientemente de la fragmentación de `pedidos`?
3. En la fragmentación vertical de `productos`, la columna `sku` aparece en **ambos** fragmentos (`v_basico` y `v_detalle`), aunque la condición de disjunción exige que las columnas no clave no se repitan. ¿Está esto mal? Justifica técnicamente por qué la duplicación de `sku` (siendo una clave alternativa, no la clave primaria) puede considerarse una decisión de diseño deliberada y correcta, y no una violación de la condición de disjunción.
4. El diseño establece que `bdd-nodo06` actuará como coordinador usando el motor Spider de MariaDB. Describe conceptualmente cómo procesaría la Consulta Distribuida 4 (de la sección D.5) en un sistema completamente distribuido: ¿cuántas subconsultas generaría el coordinador, a qué nodos se las enviaría, qué resultados intermedios recibiría y cómo los combinaría para producir la respuesta final?

5. El teorema CAP establece que ante una partición de red, el sistema debe elegir entre Consistencia y Disponibilidad. Dado el diseño completo del laboratorio (replicación en nodo01–03, fragmentación en nodo04–05, coordinador en nodo06), identifica al menos dos escenarios de fallo de red distintos y explica qué dilema CAP presentaría cada uno: ¿qué parte del sistema elegiría Consistencia y qué parte elegiría Disponibilidad?
- 

## Ejercicios prácticos

1. **Script de validación automática de correctitud.**

Escribir un único script `.sql` que verifique las tres condiciones (completitud, disjunción, reconstrucción) para las **cuatro** tablas incluidas en el plan de fragmentación, de forma automática. El script debe producir una tabla con columnas `condicion`, `tabla` y `resultado` ( `OK` / `FALLA` ). Ejecutarlo y documentar la salida completa. Modificar deliberadamente un predicado para introducir una violación de disjunción o completitud, observar cómo el script la detecta, y restaurar el predicado correcto.

2. **Propuesta de fragmentación alternativa de cuatro fragmentos.**

Proponer un diseño de fragmentación horizontal para `clientes` que use **cuatro fragmentos** (uno por valor de `region`), pensado para un escenario futuro con cuatro nodos de sharding (nodo04 a nodo07). Reescribir los predicados, identificar el nodo destino de cada uno, y ejecutar el equivalente del bloque D.3 para verificar la correctitud del diseño alternativo. Discutir las ventajas y desventajas frente al diseño de dos fragmentos actual.

3. **Análisis del grado de localidad de cada consulta simulada.**

Para cada una de las cuatro consultas distribuidas de D.5, indicar: (a) qué fragmentos necesita el coordinador, (b) si la consulta se puede resolver accediendo a un solo nodo o requiere múltiples nodos, (c) cuál de las cuatro consultas tiene el mayor grado de localidad (menor número de nodos involucrados) y cuál el menor, y (d) qué volumen estimado de datos intermedios (filas × bytes) debería transferirse entre nodos para resolver cada consulta. Apoyarse en las estadísticas del análisis D.2.

---

## Reto adicional para alumnos avanzados

Ampliar el plan de fragmentación para incluir a `productos` también en la fragmentación **horizontal**, no solo en la vertical. Dado que `productos` no tiene una columna `region`, proponer la adición de un atributo de partición (por ejemplo, `tipo_producto ENUM('hardware','mobiliario','accesorios')` u otro atributo de negocio

que se pueda derivar del campo `categoria` existente), justificar la modificación del DDL, redefinir la estrategia híbrida que combine la fragmentación vertical ya diseñada con esta nueva fragmentación horizontal, y verificar las condiciones de correctitud del esquema híbrido resultante. Incluir también el análisis de integridad referencial: si `detalle_pedidos` tiene FK hacia `productos`, ¿cómo se gestionaría esa integridad referencial en un sistema verdaderamente distribuido donde las FK no pueden cruzar nodos? ¿Qué opciones ofrece MariaDB Spider para este escenario?

---

## Criterios de evaluación para el profesor

| Criterio                                           | Peso | Indicador de logro                                                                                                                                                                                                                                                                                      |
|----------------------------------------------------|------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Análisis estadístico del esquema                   | 10%  | Se ejecutaron los tres bloques de D.2 y el estudiante interpreta correctamente la distribución de filas por región y el perfil de tamaño de las columnas TEXT; puede relacionar cada métrica con una decisión de diseño                                                                                 |
| Fragmentación horizontal: definición y correctitud | 25%  | Los predicados de <code>frag_A</code> y <code>frag_B</code> están bien definidos para las tres tablas involucradas; las tres condiciones de correctitud devuelven <code>OK</code> con la evidencia SQL; el estudiante puede explicar qué significaría un resultado <code>FALLA</code> en cada condición |
| Fragmentación vertical: definición y correctitud   | 20%  | Los dos fragmentos de <code>productos</code> están bien definidos con justificación de afinidad de columnas; las condiciones de correctitud (incluida la reconstrucción por JOIN) devuelven <code>OK</code> ; el estudiante puede explicar el rol de la PK duplicada                                    |
| Consultas distribuidas simuladas                   | 20%  | Las cuatro consultas de D.5 producen resultados correctos y no vacíos; el estudiante puede identificar qué fragmentos consultaría el coordinador para cada una y estimar el número de nodos involucrados                                                                                                |
| Documento de Diseño Distribuido                    | 15%  | El archivo <code>fase09-disenyo-distribuido.md</code> existe, es internamente coherente y cubre las siete secciones requeridas; podrá usarse como referencia sin ambigüedad en las Fases 10–16                                                                                                          |
| Comprensión conceptual (preguntas teóricas)        | 10%  | Las respuestas a las cinco preguntas usan vocabulario técnico correcto (predicado, minterm, completitud, disjunción, co-localización, coordinador, CAP) y hacen referencia explícita al esquema específico del laboratorio                                                                              |

## Preparación para la siguiente fase

La Fase 10: Replicación Física (Maestro-Esclavo) requerirá:

- El esquema `lab_bdd` idéntico y consistente en `bdd-nodo01` y `bdd-nodo02` (Fase 8).
- Snapshot `fase09-completa` tomado en ambas VMs (esta fase).
- Acceso SSH funcional desde el host hacia ambos nodos (Fase 6).
- El Documento de Diseño Distribuido disponible en `C:\LabBDD\Documentacion\fase09-disenyo-distribuido.md` como referencia: la sección 2 define que `nodo01` es el maestro y `nodo02` el esclavo, con binary log basado en GTID y replicación row-based.
- Ningún nodo adicional todavía: `bdd-nodo03`, `bdd-nodo04`, `bdd-nodo05` y `bdd-nodo06` no existen aún como VMs. Se crearán en sus respectivas fases.

En la Fase 10 se modificará la configuración de MariaDB en `bdd-nodo01` (habilitando el binary log, asignando un `server-id` único y configurando GTID) y en `bdd-nodo02` (configurando el agente de replicación esclavo y el `server-id` propio), y se verificará que los cambios escritos en el maestro se propagan automáticamente al esclavo en tiempo real.

---